Phase 10 Completion Report: Agile / SDLC Simulation

1. Phase Overview

Phase 10 of the Banking Data Governance & Data Quality Pipeline focused on simulating how Data Governance, Data Quality, Catalog, Lineage, Metadata, and OpenMetadata-related work is delivered in an Agile/Scrum environment.

The goal of this phase was to practice the non-coding delivery process used in enterprise governance programs. In real organizations, governance work is not only about creating technical pipelines, data quality checks, catalogs, or metadata assets. It also requires user stories, sprint planning, acceptance criteria, Definition of Done, test evidence, release notes, approvals, and stewardship workflows.

Phase 10 was intentionally designed as a documentation-only phase. No pipeline code, SQL validation logic, PySpark validation logic, governance artifacts, OpenMetadata assets, Docker configuration, databases, ingestion scripts, Spark jobs, or curated models were modified.

The completed project before Phase 10 already included:

PostgreSQL Banking Source System
→ Python Ingestion Layer
→ Raw Parquet Zone
→ PySpark Standardized Layer
→ Curated Warehouse Layer
→ SQL Data Quality Validation Layer
→ PySpark Data Quality Validation Layer
→ Governance Artifacts
→ OpenMetadata Integration Documentation and Templates

Phase 10 added the Agile / SDLC delivery simulation layer on top of the completed technical and governance work.

2. Phase 10 Objective

The objective of Phase 10 was to create Agile and SDLC artifacts that show how governance work would be planned, tracked, validated, released, and supported in an enterprise environment.

The phase focused on:

- Agile user stories
- Governance epics
- Sprint planning
- Sprint backlog management
- Story points

- Acceptance criteria
- Definition of Done
- Test evidence
- Release notes
- Stewardship issue tracking
- Governance issue workflow
- Data Quality issue workflow
- Ownership and escalation paths

This phase helped connect the completed technical project to real-world Agile/Scrum delivery practices expected in Data Governance, Collibra-style, Data Quality, and Metadata Management roles.

3. **Phase 10 Scope**

Phase 10 was split into two sub-phases:

- Phase 10A: Agile Governance Planning Artifacts
- Phase 10B: SDLC Evidence, Release Notes, and Stewardship Workflow

This split kept the scope disciplined and prevented the phase from becoming too large.

In Scope:

- Agile user stories
- Governance, Catalog, Lineage, Data Quality, and OpenMetadata epics
- Sprint backlog
- Story points
- Story priorities
- Story owners
- Story statuses
- Definition of Done
- Test evidence documentation
- Release notes
- Stewardship workflow
- Governance issue tracking process
- Data Quality issue tracking process

- Documentation of completed validation evidence from previous phases

Out of Scope:

- Pipeline code changes
- SQL validation logic changes
- PySpark validation logic changes
- Governance YAML artifact changes
- OpenMetadata asset changes
- New source tables
- New data models
- New Spark jobs
- New ingestion scripts
- New Docker services
- New databases
- Runtime data generation
- Runtime validation execution

4. **Phase 10A: Agile Governance Planning Artifacts**

Phase 10A focused on creating Agile planning artifacts for the governance program.

Files Created:

- agile/user_stories.md
- agile/sprint_backlog.yml

Phase 10A Work Completed:

Phase 10A created 19 Agile user stories across the following epics:

- Governance Epic
- Catalog Epic
- Lineage Epic
- Data Quality Epic
- OpenMetadata Epic

Each user story included:

- Story ID

- Epic
- Title
- Story points
- User story
- Acceptance criteria

The purpose of these stories was to simulate how governance requirements would be captured in an enterprise Agile tool such as JIRA.

Example areas covered by the stories included:

- Governance policy creation
- Business glossary definition
- Critical Data Element documentation
- Data asset cataloging
- Ownership and stewardship assignment
- Source-to-target mapping
- Lineage documentation
- SQL Data Quality validation
- PySpark Data Quality validation
- Data Quality evidence documentation
- OpenMetadata integration planning
- Metadata ingestion configuration templates

Phase 10A also created a four-sprint Scrum backlog.

The sprint backlog included:

- Sprint goals
- Stories assigned to each sprint
- Priorities
- Owners
- Status values
- Story points

The four-sprint structure simulated a realistic governance delivery plan:

Sprint 1: Governance Foundation
Sprint 2: Catalog and Lineage
Sprint 3: Data Quality
Sprint 4: OpenMetadata

Owner roles included:

- Data Governance Lead
- Data Steward
- Data Engineer
- Data Quality Engineer
- Metadata Administrator

All story statuses were marked as Done because the backlog represented work already completed across previous phases.

Phase 10A Validation Results:

Phase 10A validation confirmed:

- 19 user stories were created.
- 4 sprints were documented.
- Governance, Catalog, Lineage, Data Quality, and OpenMetadata epics were covered.
- Every story included story points and acceptance criteria.
- Sprint backlog included owners, priorities, statuses, and story points.
- Only Phase 10A Agile files were changed.
- No pipeline, governance, OpenMetadata, Docker, SQL, Spark, Python, data, logs, or README files were changed.
- Git working tree was clean after commit.

Phase 10A Commit:

9b28cb1 Phase 10A - Add Agile governance planning artifacts

5. **Phase 10B: SDLC Evidence, Release Notes, and Stewardship Workflow**

Phase 10B focused on documenting the SDLC completion evidence and release management process for the completed governance project.

Files Created:

- agile/definition_of_done.md
- agile/test_evidence.md

- agile/release_notes.md

Phase 10B Work Completed:

Phase 10B added the enterprise-style delivery evidence needed to close out the Agile / SDLC simulation layer.

The Definition of Done documented the criteria required before governance work can be considered complete. It included:

- Governance deliverable checklist
- Documentation requirements
- Validation requirements
- Review requirements
- Approval requirements
- Out-of-scope guardrails

The Definition of Done covered governance deliverables such as:

- Business glossary
- Critical Data Elements
- Governance policy
- Data catalog
- Source-to-target mapping
- Lineage mapping
- Data Quality rule catalog
- Stewardship issue records
- OpenMetadata documentation

The validation requirements included:

- SQL Data Quality validation evidence
- PySpark Data Quality validation evidence
- YAML readability validation
- OpenMetadata template validation
- Git diff review
- Clean Git status

The review requirements simulated enterprise role-based review from:

- Data Engineer
- Data Quality Engineer
- Data Steward
- Data Governance Lead
- Metadata Administrator

The approval requirements confirmed:

- Phase completion approval
- Commit completion
- No out-of-scope changes
- Evidence documentation
- Release notes completion

6. Test Evidence Documentation

The test evidence file documented validation evidence collected from completed phases.

Phase 6 SQL Data Quality Evidence:

Phase 6 evidence documented the SQL validation layer built using DuckDB over Parquet data.

The SQL validation evidence included:

- SQL validation runner
- Null checks
- Duplicate logical-key checks
- Accepted-value checks
- Referential-integrity checks
- Source-to-target reconciliation
- Control-total checks
- Exception-record queries

Final Phase 6 SQL validation result:

- Full SQL suite: 54 checks
- Failed checks: 0

Output locations documented:

- data/dq_results/sql/load_date=2026-06-16/
- logs/sql/

Phase 7 PySpark Data Quality Evidence:

Phase 7 evidence documented the PySpark-based validation framework.

The PySpark validation evidence included:

- Dataset profiling
- Column profiling
- Schema validation
- Required-column validation
- Duplicate logical-key checks
- Accepted-value checks
- Range checks
- Referential-integrity checks
- Source-to-target reconciliation
- Control-total validation
- Exception-record generation

Final Phase 7 PySpark validation result:

- All 7 curated datasets passed.
- Exception record count: 0.

Output locations documented:

- data/dq_results/pyspark/load_date=2026-06-16/
- logs/dq/

Phase 8 Governance Artifact Evidence:

Phase 8 evidence documented the manual governance artifacts created for the project.

The governance evidence included:

- Business glossary created
- 15 Critical Data Elements defined
- Governance policy created
- Data catalog created

- Source-to-target mappings created
- Lineage mappings created
- Data Quality rule catalog created
- Sample stewardship issue records created
- YAML files readable
- No pipeline logic changed

Phase 9 OpenMetadata Evidence:

Phase 9 evidence documented the OpenMetadata integration planning layer.

The OpenMetadata evidence included:

- OpenMetadata README created
- Metadata ingestion configuration templates created
- Local OpenMetadata runbook created
- No secrets committed
- Environment variable placeholders used
- No OpenMetadata installation executed
- No runtime ingestion executed
- No pipeline logic changed

Phase 10A Agile Planning Evidence:

Phase 10A evidence documented the Agile planning artifacts.

The Agile evidence included:

- 19 user stories created
- 4-sprint backlog created
- Governance, Catalog, Lineage, Data Quality, and OpenMetadata epics covered
- Owners, priorities, statuses, and story points documented

Screenshot Placeholder Section:

The test evidence file also included screenshot placeholders for future documentation, such as:

- SQL DQ validation output
- PySpark DQ validation output
- Governance artifact folder

- OpenMetadata documentation and template folder
- Agile sprint backlog

No actual screenshots were added in this phase.

7. **Release Notes**

The release notes summarized the full project capabilities delivered through Phase 10.

The release summary documented that the project now demonstrates a full Banking Data Governance & Data Quality Pipeline practice lab.

Features Delivered:

- PostgreSQL source system
- Synthetic banking source data
- Python ingestion layer
- Raw Parquet zone
- PySpark standardized layer
- Curated warehouse layer
- SQL Data Quality validation
- PySpark Data Quality validation
- Governance artifacts
- OpenMetadata integration documentation and templates
- Agile / SDLC simulation layer

Governance Capabilities Delivered:

- Business glossary
- Critical Data Elements
- Governance policy
- Ownership and stewardship definitions
- Data classifications
- Stewardship issue examples
- Governance operating rules

Data Quality Capabilities Delivered:

- SQL-based DQ checks

- PySpark-based DQ checks
- Null validation
- Duplicate validation
- Accepted-value validation
- Range validation
- Referential-integrity validation
- Source-to-target reconciliation
- Control-total validation
- Exception-record generation
- Test evidence documentation

Catalog Capabilities Delivered:

- Manual data asset catalog
- Asset descriptions
- Owners and stewards
- Classifications
- OpenMetadata catalog mapping

Lineage Capabilities Delivered:

- Source-to-target mapping
- Asset-level lineage
- Field-level lineage examples
- OpenMetadata lineage concept mapping

OpenMetadata Capabilities Delivered:

- OpenMetadata integration design
- Metadata ingestion templates
- Local deployment runbook
- Secure credential placeholder usage
- Governance platform mapping

Agile / SDLC Capabilities Delivered:

- User stories

- Epics
- Sprint backlog
- Definition of Done
- Test evidence
- Release notes
- Stewardship workflow

8. Stewardship Issue Tracking Workflow

Phase 10B documented a realistic enterprise stewardship workflow for governance and Data Quality issues.

The workflow is:

Detected → Triaged → Assigned → Remediated → Validated → Closed

Workflow Explanation:

Detected:
A governance or Data Quality issue is identified through SQL validation, PySpark validation, metadata review, catalog review, lineage review, or steward feedback.

Triaged:
The issue is reviewed for severity, business impact, affected asset, related Critical Data Element, and ownership.

Assigned:
The issue is assigned to the correct owner or resolver. The Data Steward typically owns the business review, while the Data Quality Engineer or Data Engineer owns technical investigation and remediation.

Remediated:
The assigned resolver updates the source data, transformation logic, metadata artifact, documentation, rule definition, or operational process depending on the issue type.

Validated:
The fix is validated through SQL DQ checks, PySpark DQ checks, metadata review, governance artifact review, or steward approval.

Closed:
The issue is closed after validation evidence is captured and the Data Steward or Governance Lead confirms completion.

Escalation Roles:

- Data Steward
- Data Quality Engineer

- Data Governance Lead
- Data Owner
- Platform Owner

This workflow simulates how governance and data quality issues are handled in real enterprise environments.

9. **Phase 10 Validation**

Phase 10B validation confirmed:

- agile/definition_of_done.md was created.
- agile/test_evidence.md was created.
- agile/release_notes.md was created.
- Required content and metrics were included.
- Screenshot entries were placeholders only.
- Only the three requested Phase 10B files changed.
- Phase 10A files remained unchanged.
- No pipeline files were changed.
- No SQL validation files were changed.
- No PySpark validation files were changed.
- No governance artifact files were changed.
- No OpenMetadata files were changed.
- No Docker files were changed.
- No README changes were made.
- Git working tree was clean after commit.

Phase 10B Commit:

fbd512d Phase 10B - Add SDLC evidence and release artifacts

10. **Final Phase 10 Git History**

Phase 10A Commit:

9b28cb1 Phase 10A - Add Agile governance planning artifacts

Phase 10B Commit:

fbd512d Phase 10B - Add SDLC evidence and release artifacts

Recommended Phase 10 Tag:

phase-10-complete

11. Final Phase 10 Status

Phase 10 is complete.

Final status:

- Phase 10A Agile planning artifacts: Complete
- Phase 10B SDLC evidence and release artifacts: Complete
- User stories: Complete
- Sprint backlog: Complete
- Definition of Done: Complete
- Test evidence: Complete
- Release notes: Complete
- Stewardship workflow: Complete
- Governance issue tracking process: Complete
- Data Quality issue tracking process: Complete
- Pipeline logic unchanged: Complete
- Governance artifacts unchanged: Complete
- OpenMetadata files unchanged: Complete
- SQL validation logic unchanged: Complete
- PySpark validation logic unchanged: Complete
- Git commits completed: Complete

12. Skills Practiced in Phase 10

Phase 10 helped practice the following skills:

- Agile delivery
- Scrum planning
- JIRA-style user story writing
- Epic definition
- Story-point estimation
- Sprint backlog creation

- Acceptance criteria writing
- SDLC documentation
- Definition of Done creation
- Test evidence documentation
- Release note writing
- Governance delivery tracking
- Data Quality delivery tracking
- Metadata delivery planning
- Stewardship workflow design
- Ownership and escalation modeling
- Requirement-to-release traceability
- Enterprise communication
- Audit-readiness documentation
- Git commit discipline

13. Governance and SDLC Relevance

Phase 10 is important because enterprise Data Governance programs require structured delivery and audit-ready evidence.

In real organizations, governance work is usually tracked through Agile tools such as JIRA. Teams do not simply create glossary terms, catalog assets, DQ checks, or lineage mappings without process. They define requirements, create user stories, assign owners, track progress through sprints, validate the work, collect evidence, obtain approvals, publish release notes, and support issues through stewardship workflows.

Phase 10 connected the completed technical and governance project to this enterprise delivery process.

The project now demonstrates the full lifecycle:

Requirement → User Story → Sprint Delivery → Validation → Evidence → Review → Release → Stewardship Support

This is directly relevant to Data Governance, Collibra-style, Data Quality, Metadata Management, and Agile/Scrum-based Data Engineering roles.

14. Interview Explanation

A strong interview explanation for Phase 10 would be:

“In Phase 10, I added an Agile and SDLC simulation layer to my Banking Data Governance and Data Quality Pipeline. By this point, the project already had a PostgreSQL source system, Python ingestion, PySpark processing, curated warehouse models, SQL data quality checks, PySpark data quality checks, governance artifacts, and OpenMetadata integration documentation.

The purpose of Phase 10 was to show how this type of governance work would be delivered in a real enterprise Agile environment. I created user stories across Governance, Catalog, Lineage, Data Quality, and OpenMetadata epics. I also created a sprint backlog that simulated how the work would be planned and delivered across multiple sprints.

Then I created a Definition of Done, test evidence documentation, release notes, and a stewardship issue workflow. The test evidence referenced completed SQL and PySpark validation results, governance artifact validation, OpenMetadata documentation, and Agile planning artifacts.

I intentionally kept Phase 10 documentation-only. I did not modify any pipeline code, validation logic, governance YAML, OpenMetadata files, Docker configuration, or runtime data. The goal was to practice Agile delivery, SDLC documentation, release management, and governance stewardship processes around the technical governance pipeline.”

15. Phase 10 Conclusion

Phase 10 successfully completed the Agile / SDLC Simulation layer of the Banking Data Governance & Data Quality Pipeline.

The project now demonstrates not only technical implementation, but also enterprise delivery discipline. It shows how governance, metadata, lineage, catalog, and data quality work can be planned, tracked, validated, released, and supported using Agile and SDLC practices.

After Phase 10, the project demonstrates the following capabilities:

- Data Engineering
- Data Warehousing
- SQL
- PySpark
- Python
- Data Quality
- Data Governance
- Data Catalog
- Data Lineage
- Metadata Management
- OpenMetadata

- Agile Delivery
- Scrum Planning
- SDLC Documentation
- Stewardship Workflow
- Release Management

This makes the project stronger for interview discussions because it now covers both technical delivery and enterprise governance operating process.